PORTFOLIO / 個人開発

本物の部屋を、ブラウザだけで歩ける内見体験

3D Gaussian Splatting（3DGS）で撮影した実物件を、アプリ不要・WebGL2だけで内見できるビューアです。 撮影データの前処理から配信・物件 URL の発行・閲覧計測まで、一人で企画・設計・実装・運用しました。

3D Gaussian Splatting | WebGL2 / PlayCanvas | Vanilla JavaScript | Python / NumPy | Cloudflare Workers + D1 | Google Cloud Storage | GitHub Pages

-77%
PLY 圧縮による転送量削減（59MB → 13.5MB）

0
フロントエンドの外部依存（エンジン同梱・ビルド不要）

11
自作 JS モジュール（約 5,500 行）

43 秒
公開準備ワンボタン実測（116 万スプラット）

OVERVIEW

解決したかったこと

不動産のオンライン内見は 360 度写真が主流ですが、定点からの見回しでは部屋の広さ・奥行き・動線が伝わりません。 3DGS なら「歩き回れる」体験を作れますが、研究用ツールのままでは一般の閲覧者や不動産会社の運用には使えませんでした。

Before — 3DGS を素のまま使うと

- メッシュが無いので壁をすり抜け、床にも立てない
- 撮影データが傾いていて、歩くと酔う
- 1 物件 59MB。スマホの初回表示が遅い
- 専用ビューアやアプリのインストールが前提
- 「どの物件を誰が何回見たか」が分からない

After — このビューアでは

- スプラットから自動生成したコリジョンで、目線を床に固定して歩ける
- 前処理で傾きを自動補正。壁と床がワールド軸に揃う
- 圧縮 PLY と gzip 配信で 13.5MB。見た目の劣化はほぼ無し
- URL を開くだけ。PC / スマホ対応、ジョイスティック UI
- 物件ごとの固有 URL・QR・流入元別の閲覧集計まで一式

FEATURES

主な機能

閲覧者向けのビューア、不動産会社向けの管理ツール、プラットフォーム層の 3 つで構成しています。

ウォークモード

目線の高さを床に自動追従させ、WASD / ジョイスティックで実際に部屋を歩く感覚で移動。壁・家具にはコリジョンで止まります。

自動コリジョン生成

メッシュを持たない 3DGS から、ボクセルと床・天井高さマップを自前で生成。ガラスや鏡など検出できない面は管理ツールで手動箱を追加できます。

テレポート＆場所一覧

部屋ごとのテレポートポイントに近づくと「ここへ移動」を表示。右上のメニューから任意の部屋へ直接ジャンプもできます。

360 度パノラマ連携

3DGS で撮れない部屋（風呂・トイレなど）は 360 度写真で補完。自前 WebGL の Panini 投影で室内を自然に見せ、元の部屋に戻れます。

物件固有 URL と QR

Cloudflare Worker が `/p/{id}` で OGP 付きページを配信。QR コードも生成し、SNS / ポータル / チラシなど流入元別に閲覧数を集計します。

公開準備ワンボタン

撮影フォルダを選ぶだけで、水平化 → コリジョン生成 → 圧縮 → GCS アップロードを順に実行。ログをリアルタイム表示し、終わったら即ビューアで確認できます。

TECH STACK

技術スタックと選定理由

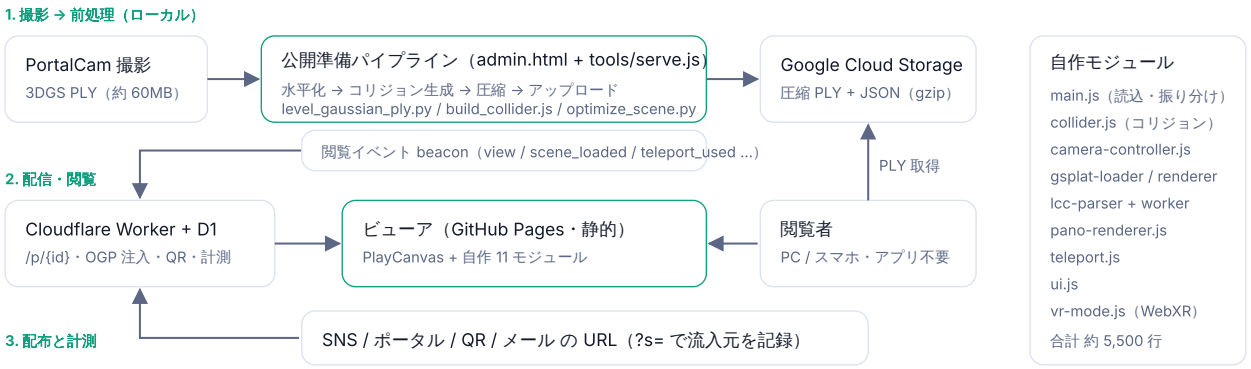
「本番が突然壊れない」「ビルド無しで誰でも配信できる」を優先し、依存を最小にしています。

領域	採用技術	選定理由
レンダリング	PlayCanvas 2.7.4（`GSplatComponent`）	3DGS をネイティブ対応。CDN 参照ではなく `vendor/` に同梱・バージョン固定し、破壊的更新と CDN 障害を遮断
フロントエンド	Vanilla JavaScript（ES Module 無し）	ビルド工程を持たず `<script>` 直読みで動かす。GitHub Pages にそのまま置ける
コリジョン	自前ボクセル / 高さマップ（SVO 対応）	3DGS はメッシュを持たず物理エンジンが使えないため、スプラット密度から自作
360 度表示	自前 WebGL（フラグメントシェーダ）	Panini 投影を固定しオンデマンド描画で省電力。ライブラリ非依存
前処理	Python + NumPy / Node.js CLI	PLY の傾き補正・圧縮・コリジョン生成をコマンド化し、管理ツールからも呼べるように
配信	Google Cloud Storage + GitHub Pages	静的配信のみで運用コストを最小化。Cache-Control と gzip 転送を使い分け
プラットフォーム	Cloudflare Workers + D1（SQLite）	エッジで OGP 注入・期限判定・閲覧イベント記録。ビューア本体は静的のまま
計測	GA4 + 自前 beacon	読込時間・テレポート利用・パノラマ遷移などをイベント化し、流入元別に集計

ARCHITECTURE

システム構成

撮影データは前処理パイプラインを通して GCS に置き、ビューアはそれを読むだけ。物件の管理と計測は Worker 層に分離しています。



main.js
URL / ファイル / フォルダ読込の振り分け、コンパニオン適用、許可リスト、計測

collider.js
SVO / 高さマップ / 手動箱による移動制限・床追従・スイープ判定（1,660 行）

camera-controller.js
入力→カメラ姿勢。ウォーク / 自由飛行、テレポート、読込直後の床スナップ

pano-renderer.js
360 度画像を Panini 投影で描画する自前 WebGL ビューア

lcc-parser.js
XGRIDS LCC 形式を Web Worker で .splat に変換

worker/src/index.js
物件レジストリ・OGP 注入・QR 生成・閲覧集計 API（Cloudflare Workers）

ENGINEERING

技術的な工夫と乗り越えた課題

実物件のデータで運用して初めて見つかった問題を、計測しながら一つずつ潰しました。

01 メッシュの無い 3DGS に「歩ける床と壁」を作る

3DGS は点群に近いデータで物理エンジンが使えません。スプラットの位置と半径からボクセルグリッドを生成し、XZ 平面の壁マスクと床・天井の高さマップに落とし込む自前コリジョンを実装しました。 薄い壁が潰れないよう約 2.5cm/ボクセルを狙い、メモリ上限に合わせて解像度を 128〜192 で可変にしています。

ノイズ対策として、全点の min/max ではなく上下 0.3% を切り捨てたパーセンタイル bbox を使い、連結成分フィルタで孤立した浮遊ノイズを除去。 ガラスや鏡のように撮影に写らない面は、管理ツールから手動コリジョン箱を置けるようにしました。

結果: **壁抜け無し**で部屋を歩ける。狭い廊下の通過可否は `collider_reach.js` で到達面積を数値化して回帰テスト

02 窓の外のスプラットで廊下が「壁」になる問題

運用中に特定の物件だけ廊下を通れない症状が出ました。原因は、窓越しに写り込んだ屋外の建物や空のスプラットが 0.3% を超え、bbox が数十 m に膨らんで 1 ボクセルが 20cm 超になっていたこと。

対策として 0.25m の密度格子で「面として密なセル」の最大連結成分（床・壁・天井の殻）を本体とみなし、そこから 0.75m 以内の成分だけを範囲に含めるように変更。壁の膨らみ量も物理長 7.5cm で頭打ちにしました。

結果: 影響のあった **3 物件すべて**で再生成のみで解消。既存データは「コリジョンを作り直す」ボタン一つで更新

03 撮影データの傾きを自動補正する

撮影機が出力する PLY は床が水平とは限らず、傾いたまま歩くと酔います。各スプラットの最短スケール軸を面法線とみなし、壁 2 方向と床・天井の法線クラスタを反復推定して初期回転を求めました。 さらに軸方向 1cm ヒストグラムの集中度を最大化する座標降下で微調整しています。

位置だけでなく回転クォータニオンも同時に回すことで表示を壊さず、処理済みファイルにはヘッダーに印を付けてパイプラインの二重実行を防ぎました。

結果: 水平化を**人手ゼロ**で公開準備に組み込み

04 初回表示を速くする：転送量 -77%

生の PLY は 1 スプラット 68 バイト、87 万スプラットで 59MB ありました。splat-transform の圧縮 PLY 形式（16 バイト/スプラット）に変換して 13.5MB に。 同一視点で描画を比較し、64 ブロック平均の絶対差 0.4/255・最大 2/255 と画質劣化がほぼ無いことを確認してから採用しました。

コンパニオン JSON は gzip 転送で 2.25MB → 0.19MB。本体は immutable 長期キャッシュ、JSON は短期キャッシュと Cache-Control を使い分けています。 SOGS 形式も検証しましたが、この素材では 3.6MB しか差が無く改修コストに見合わないと判断して見送りました。

結果: **59MB → 13.5MB**。拡張子は .ply のままでビューア側は無改修

05 「特定の人だけ画面が真っ黒」の障害対応

JS を更新した後、以前に訪問したことのある人だけ読み込みが止まる障害が起きました。新しい HTML と古いキャッシュ JS が混在し、モジュール間の API 不一致で落ちていたのが原因です。

全スクリプトに `?v=日付` のキャッシュバスタを付け、更新手順を README に明文化。合わせて `?url=` で読み込める先を GCS バケットと同一オリジンに制限する許可リストも入れ、任意 URL 読込によるリスクを塞ぎました。

結果: 再発ゼロ。**運用手順をドキュメント化**して属人化を防止

06 不動産会社が運用できる形にする

技術デモで終わらせないため、物件登録から配布・計測までを一本化しました。Cloudflare Worker が物件 ID ごとに OGP 付きページと QR を配信し、掲載期限や成約時のアーカイブも管理 API で操作できます。

流入元は URL パラメータで記録し、SNS・ポータル・店頭 QR・メールのどこから何回見られたかを日別に集計。管理ツール側は「フォルダを選んで実行」だけで公開でき、116 万スプラットの物件で 43 秒で完了します。

結果: 撮影から公開まで**エンジニア不要**のワークフロー

TIMELINE

開発の経緯

個人開発として企画から運用まで担当。約 4 か月・50 コミットで、技術検証から不動産会社が使えるプラットフォームまで育てました。

2026.06

技術検証

PlayCanvas の 3DGS ネイティブ対応を検証し、PLY / .splat / LCC の読込と基本的なカメラ操作を実装。

2026.07

集中開発：歩ける内見へ

自前コリジョンとウォークモード、テレポート、360 度パノラマ連携、スマホ UI を実装。傾き補正と PLY 圧縮の前処理ツールを整備。

2026.08

運用開始と安定化

実物件での公開を開始。キャッシュ混在障害の対応、許可リスト、OGP / GA4 計測を追加。

2026.10

プラットフォーム化

Cloudflare Workers + D1 で物件レジストリと閲覧集計を構築。公開準備をワンボタン化し、窓外スプラットによるコリジョン不良を修正。

ROLE

担当範囲

企画・要件定義

360 度写真の限界と不動産会社の運用フローを踏まえ、「歩ける内見」と「エンジニア不要の公開手順」を要件に設定。

設計・実装

フロントエンド 11 モジュール、前処理 CLI、ローカル開発サーバー、Worker API、D1 スキーマまで全レイヤーを実装。

検証・運用

画質差・転送量・到達面積を数値で比較して判断。障害の原因究明と再発防止策、README / 設計ドキュメントの整備。

デモを開く　GitHub: ken1055/VR_naiken　README　ARCHITECTURE

VR 内見ビューア — 3D Gaussian Splatting による Web 内見体験。個人開発プロジェクト。